
SEZIONE 1: DOMANDE TEORICHE

Domanda 1.1 - Ricorrenze e Metodo di Sostituzione (6 punti)

Consegna: Si dimostri che la ricorrenza che segue ha soluzione $T(n) = \Theta(n)$:

$$T(n) = (2/3)T(n-1) + 2n$$

Domanda 1.2 - Alberi Binari di Ricerca (7 punti)

Consegna: Dare la definizione di albero binario di ricerca. Specificare l'albero ottenuto inserendo, con la procedura vista a lezione, a partire da un albero vuoto, i nodi aventi le seguenti chiavi: 10, 5, 3, 15, 17, 12. Si supponga che dall'albero così ottenuto si cancelli il nodo con chiave 5 e si indichi l'albero ottenuto. Sia per gli inserimenti che per la cancellazione, motivare sinteticamente il risultato ottenuto.

SEZIONE 2: DIVIDE ET IMPERA

Esercizio 2.1 - Indici Stabili (10 punti)

Consegna: Dato un array $A[1..n]$ di interi, un indice i si dice stabile se $A[i] = i$. Realizzare una procedura $\text{stab}(A, n)$ che, dato in input un array $A[1..n]$ di interi distinti ordinato in modo crescente, ritorna un indice stabile, se esiste, e ritorna 0 altrimenti. Dimostrarne la correttezza e valutarne la complessità.

Esercizio 2.2 - Gap in Array (9 punti)

Consegna: Dato un array di interi $A[1..n]$, chiamiamo gap un indice $i \in [1, n)$ tale che $A[i+1] - A[i] > 1$.

1. Mostrare per induzione su n che un array $A[1..n]$ tale che $A[n] - A[1] \geq n$ (quindi $n \geq 2$) contiene almeno un gap.
2. Fornire lo pseudocodice di una procedura ricorsiva divide et impera gap che dato un array $A[1..n]$ tale che $A[n] - A[1] \geq n$ restituisce un gap in A .
3. Valutare la complessità della funzione, utilizzando il master theorem.

Soluzione:

Parte 1: Dimostrazione per Induzione

Vogliamo dimostrare che se $A[n] - A[1] \geq n$, allora esiste almeno un gap.

Caso base ($n = 2$): Se $A[2] - A[1] \geq 2$, allora $A[2] - A[1] > 1$, quindi $i = 1$ è un gap. ✓

Passo induttivo ($n > 2$): Assumiamo la proprietà vera per array di lunghezza $< n$.

Consideriamo $A[1..n]$ con $A[n] - A[1] \geq n$.

Caso 1: Se $A[n] - A[n-1] > 1$, allora $i = n-1$ è un gap. ✓

Caso 2: Se $A[n] - A[n-1] \leq 1$, allora:

$$\begin{aligned} A[n-1] - A[1] &= (A[n] - A[1]) - (A[n] - A[n-1]) \\ &\geq n - 1 \end{aligned}$$

Per ipotesi induttiva, $A[1..n-1]$ contiene un gap, che è anche un gap per $A[1..n]$. ✓

Parte 2: Pseudocodice

```
1 gap(A, n)
2   return gap_rec(A, 1, n)

3 gap_rec(A, p, r)
4   // Precondizione:  $p < r$  e  $A[r] - A[p] \geq r - p + 1$ 
5   if  $r - p = 1$ 
6       return p //  $A[r] - A[p] > 1$  per precondizione
7    $q = \lfloor (p + r)/2 \rfloor$ 
8   if  $A[q] - A[p] \geq q - p + 1$ 
9       return gap_rec(A, p, q)
10  else
11      return gap_rec(A, q, r)
```

Dimostrazione di Correttezza:

Invariante: Se la precondizione vale, la funzione ritorna un gap.

Caso base: Se $r - p = 1$, allora per precondizione $A[r] - A[p] \geq 2$, quindi $A[r] - A[p] > 1$, e p è un gap.

Passo ricorsivo: Sia $q = \lfloor (p + r)/2 \rfloor$.

Caso 1: Se $A[q] - A[p] \geq q - p + 1$:

- La precondizione vale per $[p, q]$
- Per ipotesi ricorsiva, $\text{gap_rec}(A, p, q)$ ritorna un gap

Caso 2: Se $A[q] - A[p] < q - p + 1$:

- Allora $A[r] - A[q] = (A[r] - A[p]) - (A[q] - A[p]) \geq (r - p + 1) - (q - p) = r - q + 1$
- La preconditione vale per $[q, r]$
- Per ipotesi ricorsiva, $\text{gap_rec}(A, q, r)$ ritorna un gap

Parte 3: Analisi di Complessità

La ricorrenza è:

$$T(n) = T(n/2) + \theta(1)$$

Per il Master Theorem (caso 2 con $k=0$):

$$T(n) = \theta(\log n)$$

SEZIONE 3: PROGRAMMAZIONE DINAMICA

Esercizio 3.1 - Ricorrenza su Stringhe (9 punti)

Consegna: Data una stringa $X = (x_1, x_2, \dots, x_n)$, si consideri la seguente quantità $\ell(i, j)$, definita per $1 \leq i \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } i = j - 1 \\ 2 + \ell(i+1, j-1) & \text{se } (i < j-1) \wedge (x_i = x_j) \\ \sum_{k=i}^{j-1} [\ell(i, k) + \ell(k+1, j)] & \text{se } (i < j-1) \wedge (x_i \neq x_j) \end{cases}$$

(a) Scrivere una coppia di algoritmi $\text{INIT_L}(X)$ e $\text{REC_L}(X, i, j)$ per il calcolo memoizzato di $\ell(1, n)$.

(b) Determinarne la complessità al caso migliore $T_{\text{best}}(n)$, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

Esercizio 3.2 - Longest Common Substring (11 punti)

Consegna: Una longest common substring di due stringhe X e Y è una sottostringa di X e di Y di lunghezza massima. Si vuole progettare un algoritmo efficiente per calcolare la lunghezza di una longest common substring. Per semplicità si assuma che entrambe le stringhe di input abbiano stessa lunghezza n .

- (a) Qual è la complessità dell'algoritmo esaustivo che analizza tutte le possibili sottostringhe comuni?
- (b) Assumendo di conoscere un algoritmo che determina se una stringa di m caratteri è sottostringa di un'altra stringa di n caratteri in tempo $O(m+n)$, come si può modificare l'algoritmo del punto precedente per renderlo più efficiente?
- (c) Progettare un algoritmo di programmazione dinamica più efficiente di quello del punto precedente. Sono richiesti relazione di ricorrenza sulle lunghezze (senza dimostrazione) e algoritmo bottom-up.
-

SEZIONE 4: ALBERI BINARI DI RICERCA

Esercizio 4.1 - avgTree (9 punti)

Consegna: Realizzare una funzione `avgTree(T)` che dato un albero binario T con chiavi numeriche, verifica se, per ogni nodo che abbia discendenti, la chiave del nodo è maggiore o uguale della media delle chiavi dei discendenti e ritorna conseguentemente un valore booleano (la radice dell'albero è `T.root` e ogni nodo x ha i campi `x.left`, `x.right` e `x.key`).

Esercizio 4.2 - Arricchimento BST con Numero Foglie (9 punti)

Consegna: Realizzare un arricchimento degli alberi binari di ricerca che permetta di ottenere per ogni nodo x , in tempo costante, il numero delle foglie nel sottoalbero radicato in x .

Indicare quali campi occorre aggiungere ai nodi. Fornire il codice per la funzione `leaves(x)` che restituisce il numero delle foglie nel sottoalbero radicato in x e per la procedura di inserimento di un nodo `insert(T, z)`. Per entrambe valutare la complessità.

SEZIONE 5: MAX-HEAP

Esercizio 5.1 - Intersezione di Min-Heap (9 punti)

Consegna: Realizzare una funzione `intersect(A1, A2, n)` che dati due array di interi $A1$ e $A2$, organizzati a min-heap, con capacità n , restituisce un nuovo array A , ancora organizzato a min-heap, che contiene l'intersezione dei valori contenuti in $A1$ e $A2$. Nel caso gli array contengano più occorrenze dello stesso valore v , l'intersezione mantiene il numero minimo di occorrenze di v (ad es. se $A1$ contiene i valori $1,2,2,2$ e $A2$ contiene i valori $1,1,2,2$ allora A conterrà $1,2,2$). Valutarne la complessità.

SEZIONE 6: ALGORITMI GREEDY

Esercizio 6.1 - Minimizzazione Tempo Completamento (10 punti)

Consegna: Abbiamo n programmi da eseguire sul nostro computer. Ogni programma j , dove $j \in \{1, 2, \dots, n\}$, ha lunghezza l_j , che rappresenta la quantità di tempo richiesta per la sua esecuzione.

Dato un ordine di esecuzione $\sigma = j_1, j_2, \dots, j_n$ dei programmi (cioè, una permutazione di $\{1, 2, \dots, n\}$), il tempo di completamento $C_{ji}(\sigma)$ del j_i -esimo programma è dato quindi dalla somma delle lunghezze dei programmi $j_1, j_2, \dots, j_i$.

L'obiettivo è trovare un ordine di esecuzione σ che minimizza la somma dei tempi di completamento di tutti i programmi, cioè $\sum_i C_{ji}(\sigma)$.

(a) Dare un semplice algoritmo greedy per questo problema, e valutarne la complessità.

(b) Dimostrare la proprietà di scelta greedy dell'algoritmo del punto (a), cioè che esiste un ordine di esecuzione ottimo σ^* che contiene la scelta greedy.

Esercizio 6.2 - Selezione Attività (9 punti)

Consegna: Si considerino n attività $\{a_1, a_2, \dots, a_n\}$, ciascuna caratterizzata da tempo di inizio s_i e tempo di fine f_i (con $s_i < f_i$). Due attività a_i e a_j sono compatibili se i loro intervalli temporali non si sovrappongono ($f_i \leq s_j$ oppure $f_j \leq s_i$).

Vogliamo selezionare un insieme massimale di attività mutualmente compatibili.

(a) Scrivere il codice di un algoritmo greedy che risolve il problema in tempo lineare, assumendo che le attività siano già ordinate per tempo di fine crescente ($f_1 \leq f_2 \leq \dots \leq f_n$).

(b) Dimostrare la proprietà di scelta greedy per l'algoritmo del punto (a).
